

DAY 1: Mobile OTP verification, BACKEND

E-Learning Platform (Spring Boot 3.2.5, Java 17, MySQL). **9 new files + 3 tiny edits**. Nothing existing is rewritten, and with `otp.required=false` (the default) the app behaves exactly as before.

1. OTP explained from zero

OTP = One-Time Password. The server makes a random 6 digit number, sends it to the mobile number the user typed, and asks the user to type it back. If it matches, the user really owns that number. It is used once and dies after 5 minutes.

Why bother? Anyone can type any mobile number in a form. The OTP proves the person holds the phone. That is the SME's requirement for customers and instructors.

How it flows in your project

1. User registers (`POST /api/register` or `POST /api/instructor/register`). The account is created as normal.
2. `OtpService.markPending(user)` adds a row in the new table `phone_verifications` with `verified = false`. Pending means "must verify before login".
3. The browser opens `/verify-phone`, which calls `POST /api/otp/send`. The server makes the code, stores **only a salted hash of it** (never the code itself), and passes the text to an `SmsSender`.
4. The user types the code. `POST /api/otp/verify` compares hashes. On success the row becomes `verified = true`.
5. `POST /api/login` calls `OtpService.requireVerified(user)`. A pending user gets **403 "Please verify your mobile number to continue"** and the login page sends them to `/verify-phone`.

Who counts as verified? Everyone who has **no row** in `phone_verifications`: all your old users, the seeded admin, and Google sign-in users. Only brand-new customers and instructors get a pending row. That is why switching OTP on cannot lock out existing accounts.

Safety rules built in (each one is a real attack it stops)

Rule	Stops
6 digits from <code>SecureRandom</code>	guessing the next code
only the hash is stored, with a random salt	a database leak revealing codes
expires after 5 minutes	old codes being used later
5 wrong tries, then the code is dead	brute force of the 1,000,000 combinations
30 seconds between sends, 5 sends per hour per email, per-IP limits	SMS bombing someone's phone and burning your Twilio money
the "send" answer is always the same sentence, even for unknown emails	finding out which emails are registered
<code>otp.dev-expose-code=true</code> makes the app refuse to start in the <code>prod</code> profile	leaking codes in production

SmsSender and the two providers. `SmsSender` is a tiny interface with one method, `send(number, text)`. Two classes implement it and Spring picks one from `otp.provider`:

- `dev` (default): **prints the code in the backend console** (the phone number is masked). Perfect for development and for demos without any SMS account.
- `twilio`: sends a real SMS using Twilio's REST API (plain Java `HttpClient`, no extra Maven dependency).

2. Honest status

- The OTP logic was compiled with real Java and run against fake repositories: **33 checks, 0 failures** (hashing, expiry, 5-try lock, resend gap, hourly limits, generic answers, E.164 formatting, old users counted as verified).
- The whole screen flow was tested in a real browser against a pretend backend: **20 checks, 0 failures**.
- **Not tested here:** Spring start-up wiring, MySQL table creation, and a real Twilio SMS (this environment has no Maven downloads and no Twilio account). Compile errors from a typo or missing import are possible. Paste the exact error and it is a one-minute fix.

3. Files

#	File	Purpose
1	<code>model/PhoneVerification.java</code>	table <code>phone_verifications</code> (created by <code>ddl-auto=update</code>)
2	<code>repository/PhoneVerificationRepo.java</code>	<code>findById</code>
3	<code>exception/PhoneNotVerifiedException.java</code>	the 403 answer
4-6	<code>sms/SmsSender</code> , <code>ConsoleSmsSender</code> , <code>TwilioSmsSender</code>	the sending
7	<code>dto/OtpDtos.java</code>	request and response shapes
8	<code>service/OtpService.java</code>	all the rules
9	<code>controller/OtpController.java</code>	<code>/api/otp/send</code> , <code>/verify</code> , <code>/status</code>

4. Edits to existing files (3 small ones)

4a. `controller/AuthController.java`

Add this field under the other fields. It is optional (`required = false`), so nothing breaks if the bean is missing:

```
@Autowired(required = false)
private OtpService otpService;
```

and the import `import com.examly.springapp.service.OtpService;`

In `register(...)`, after `User saved = userService.register(user);` add:

```

    if (otpService != null) {
        otpService.markPending(saved);
    }

```

In `login(...)`, right after `loginAttempts.recordSuccess(key)`; add:

```

    if (otpService != null) {
        otpService.requireVerified(user);    // throws 403 when the mobile number is not verified yet
    }

```

4b. `config/SecurityConfig.java`: add this line in the `permitAll` area, above `.anyRequest().authenticated()` (Spring uses the **first** matching rule):

```

.requestMatchers("/api/otp/**").permitAll()

```

4c. `src/main/resources/application.properties`: append

```

# ---- Mobile OTP (Day 1) ----
# required=true makes new customers and instructors verify their mobile number before they can log in.
# It is false by default so the original register/login behaviour (and the hidden tests) stay unchanged.
otp.required=${OTP_REQUIRED:false}
# dev = the code is printed in the backend console. twilio = real SMS through Twilio.
otp.provider=${OTP_PROVIDER:dev}
# only for demos in dev mode: the API also returns the code so the page can show it. Must be false in prod.
otp.dev-expose-code=${OTP_DEV_EXPOSE_CODE:false}
otp.country-code=${OTP_COUNTRY_CODE:+91}
twilio.account-sid=${TWILIO_ACCOUNT_SID}
twilio.auth-token=${TWILIO_AUTH_TOKEN}
twilio.from-number=${TWILIO_FROM_NUMBER}

```

5. Run it (dev mode, no SMS needed)

Start the backend with these environment variables (PowerShell shown; use `export` on Linux/Mac):

```

$env:OTP_REQUIRED="true"
$env:OTP_DEV_EXPOSE_CODE="true"

```

Then test with curl (or Swagger):

```

curl -X POST localhost:8080/api/register -H "Content-Type: application/json" -d "
{"username\":\"asha\", \"email\":\"asha@mail.com\", \"password\":\"Secret@123\", \"mobileNumber\":\"9876543210\", \"role\":\"customer\"}"
curl -X POST localhost:8080/api/login -H "Content-Type: application/json" -d "
{"email\":\"asha@mail.com\", \"password\":\"Secret@123\"}"

```

The login must answer **403 "Please verify your mobile number to continue"**. Then:

```

curl -X POST localhost:8080/api/otp/send -H "Content-Type: application/json" -d "
{"email\":\"asha@mail.com\"}"

```

The answer contains `devCode` (and the backend console prints the code). Verify:

```

curl -X POST localhost:8080/api/otp/verify -H "Content-Type: application/json" -d "
{"email\":\"asha@mail.com\", \"code\":\"123456\"}"

```

Now the login works.

6. Switching to real SMS with Twilio

1. Create an account at [twilio.com](https://www.twilio.com) (free trial). Verify your own phone number when asked.
2. In the Twilio Console get a trial phone number (Phone Numbers, Get a number). Copy your **Account SID** and **Auth Token** from the Console home page.
3. **Trial limit:** a trial account can only text numbers you have added under *Verified Caller IDs*. Add every phone that will be used in the demo. Messages start with "Sent from your Twilio trial account".
4. Set environment variables (never put them in a file you commit):

```
$env:OTP_PROVIDER="twilio"
$env:OTP_REQUIRED="true"
$env:OTP_DEV_EXPOSE_CODE="false"
$env:OTP_COUNTRY_CODE="+91"
$env:TWILIO_ACCOUNT_SID="ACxxxxxxxx"
$env:TWILIO_AUTH_TOKEN="xxxxxxxx"
$env:TWILIO_FROM_NUMBER="+1xxxxxxxxx"
```

1. Restart and register with a verified number. The SMS should arrive in a few seconds.

Honest warning about India: sending SMS to Indian numbers can need extra sender registration (DLT) for production, and trial delivery is not guaranteed. This was **not tested** against real Twilio. If SMS does not arrive during the demo, switch back to `OTP_PROVIDER=dev` and show the code from the console; the logic is identical. Rotate the Auth Token if it ever appears in a screenshot or chat.

7. Troubleshooting

Symptom	Cause and fix
Login never asks for OTP	<code>OTP_REQUIRED</code> is not <code>true</code> , or the user was created before OTP was switched on (old users count as verified)
404 on <code>/api/otp/send</code>	<code>OtpController</code> missing, or the package is not under <code>com.examly.springapp</code>
401/403 on <code>/api/otp/status</code>	the <code>permitAll</code> line is below a stricter rule; move it up
Refusing to start: <code>otp.dev-expose-code</code>	you are in the <code>prod</code> profile; set <code>OTP_DEV_EXPOSE_CODE=false</code>
Twilio answers 21608 or 21211	the number is not verified (trial) or not in +country format
429 on send	the 30 s gap or the hourly limit; wait

8. Complete code

`model/PhoneVerification.java` (new file)

```
package com.examly.springapp.model;

import jakarta.persistence.*;

/**
 * Phone verification state of one account (customer or instructor). A row with verified = false means
```

```

* "this account must verify its mobile number before it can log in". Accounts without a row (all accounts
* created before OTP existed, admins, Google logins) count as verified. The real code is never stored,
* only a salted hash of it.
*/
@Entity
@Table(name = "phone_verifications")
public class PhoneVerification {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false, unique = true)
    private Long userId;

    private boolean verified;

    @Column(length = 64)
    private String codeHash;

    @Column(length = 32)
    private String salt;

    /** When the current code stops working (milliseconds since 1970). */
    private long expiresAtMs;

    /** Wrong tries on the current code. */
    private int attempts;

    private long lastSentAtMs;
    private long verifiedAtMs;

    public PhoneVerification() {}

    public PhoneVerification(Long userId, boolean verified) {
        this.userId = userId;
        this.verified = verified;
    }

    public Long getId() { return id; }
    public Long getUserId() { return userId; }
    public boolean isVerified() { return verified; }
    public void setVerified(boolean verified) { this.verified = verified; }
    public String getCodeHash() { return codeHash; }
    public void setCodeHash(String codeHash) { this.codeHash = codeHash; }
    public String getSalt() { return salt; }
    public void setSalt(String salt) { this.salt = salt; }
    public long getExpiresAtMs() { return expiresAtMs; }
    public void setExpiresAtMs(long expiresAtMs) { this.expiresAtMs = expiresAtMs; }
    public int getAttempts() { return attempts; }
    public void setAttempts(int attempts) { this.attempts = attempts; }
    public long getLastSentAtMs() { return lastSentAtMs; }
    public void setLastSentAtMs(long lastSentAtMs) { this.lastSentAtMs = lastSentAtMs; }
    public long getVerifiedAtMs() { return verifiedAtMs; }
    public void setVerifiedAtMs(long verifiedAtMs) { this.verifiedAtMs = verifiedAtMs; }
}

```

repository/PhoneVerificationRepo.java (new file)

```

package com.examly.springapp.repository;

import com.examly.springapp.model.PhoneVerification;
import java.util.Optional;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;

/** Data access for phone verification rows. */
@Repository
public interface PhoneVerificationRepo extends JpaRepository<PhoneVerification, Long> {

```

```
Optional<PhoneVerification> findByUserId(Long userId);  
}
```

exception/PhoneNotVerifiedException.java (new file)

```
package com.examly.springapp.exception;  
  
import org.springframework.http.HttpStatus;  
  
/**  
 * Raised at login when the password is right but the mobile number has not been verified with an OTP yet  
 * (answered with 403). The message is fixed and safe to show; the frontend looks for it to open the  
 * "Verify your number" page.  
 */  
public class PhoneNotVerifiedException extends AppException {  
    public static final String MESSAGE = "Please verify your mobile number to continue";  
  
    public PhoneNotVerifiedException() {  
        super(HttpStatus.FORBIDDEN, MESSAGE);  
    }  
}
```

sms/SmsSender.java (new file)

```
package com.examly.springapp.sms;  
  
/** Sends one text message. Implementations: ConsoleSmsSender (development) and TwilioSmsSender. */  
public interface SmsSender {  
    /** @param toE164 phone number with country code, for example +919876543210 */  
    void send(String toE164, String message);  
}
```

sms/ConsoleSmsSender.java (new file)

```
package com.examly.springapp.sms;  
  
import org.slf4j.Logger;  
import org.slf4j.LoggerFactory;  
import org.springframework.boot.autoconfigure.condition.ConditionalOnProperty;  
import org.springframework.stereotype.Component;  
  
/**  
 * Development sender: no SMS leaves the machine, the message is printed in the backend console instead.  
 * Active when otp.provider is "dev" (the default).  
 */  
@Component  
@ConditionalOnProperty(name = "otp.provider", havingValue = "dev", matchIfMissing = true)  
public class ConsoleSmsSender implements SmsSender {  
    private static final Logger log = LoggerFactory.getLogger(ConsoleSmsSender.class);  
  
    @Override  
    public void send(String toE164, String message) {  
        String masked = toE164.length() > 4 ? "*****" + toE164.substring(toE164.length() - 4) : "*****";  
        log.info("DEV SMS (not really sent) to {}: {}", masked, message);  
    }  
}
```

sms/TwilioSmsSender.java (new file)

```
package com.examly.springapp.sms;  
  
import com.examly.springapp.exception.ApiCommunicationException;  
import java.net.URI;
```

```

import java.net.URLEncoder;
import java.net.http.HttpClient;
import java.net.http.HttpRequest;
import java.net.http.HttpResponse;
import java.nio.charset.StandardCharsets;
import java.time.Duration;
import java.util.Base64;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.boot.autoconfigure.condition.ConditionalOnProperty;
import org.springframework.stereotype.Component;

/**
 * Sends the SMS through Twilio's REST API (Programmable Messaging). No extra library is needed: it is one
 * HTTPS form post with Basic authentication (account SID and auth token). Active when otp.provider=twilio.
 * The three Twilio values come from environment variables and must never be written in a file.
 */
@Component
@ConditionalOnProperty(name = "otp.provider", havingValue = "twilio")
public class TwilioSmsSender implements SmsSender {
    private static final Logger log = LoggerFactory.getLogger(TwilioSmsSender.class);

    private final String accountSid;
    private final String authToken;
    private final String fromNumber;
    private final String baseUrl;
    private final HttpClient http = HttpClient.newBuilder().connectTimeout(Duration.ofSeconds(10)).build();

    public TwilioSmsSender(@Value("${twilio.account-sid}") String accountSid,
                          @Value("${twilio.auth-token}") String authToken,
                          @Value("${twilio.from-number}") String fromNumber,
                          @Value("${twilio.base-url:https://api.twilio.com}") String baseUrl) {
        if (accountSid.isBlank() || authToken.isBlank() || fromNumber.isBlank()) {
            throw new IllegalStateException(
                "otp.provider=twilio needs TWILIO_ACCOUNT_SID, TWILIO_AUTH_TOKEN and
TWILIO_FROM_NUMBER");
        }
        this.accountSid = accountSid;
        this.authToken = authToken;
        this.fromNumber = fromNumber;
        this.baseUrl = baseUrl.endsWith("/") ? baseUrl.substring(0, baseUrl.length() - 1) : baseUrl;
    }

    @Override
    public void send(String toE164, String message) {
        String form = "To=" + enc(toE164) + "&From=" + enc(fromNumber) + "&Body=" + enc(message);
        String basic = Base64.getEncoder().encodeToString((accountSid + ":" +
authToken).getBytes(StandardCharsets.UTF_8));
        HttpRequest req = HttpRequest.newBuilder(URI.create(baseUrl + "/2010-04-01/Accounts/" + accountSid
+ "/Messages.json"))
            .timeout(Duration.ofSeconds(15))
            .header("Content-Type", "application/x-www-form-urlencoded")
            .header("Authorization", "Basic " + basic)
            .POST(HttpRequest.BodyPublishers.ofString(form))
            .build();

        try {
            HttpResponse<String> res = http.send(req, HttpResponse.BodyHandlers.ofString());
            if (res.statusCode() / 100 != 2) {
                // the body names the reason (for example an unverified trial number); it holds no secret
                log.warn("Twilio refused the SMS: status {} body {}", res.statusCode(), res.body());
                throw new ApiCommunicationException("Twilio responded with status " + res.statusCode(),
null);
            }
        } catch (ApiCommunicationException e) {
            throw e;
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
            throw new ApiCommunicationException("Twilio request was interrupted", e);
        } catch (Exception e) {
            throw new ApiCommunicationException("Twilio request failed", e);
        }
    }
}

```

```

    }
}

private static String enc(String s) {
    return URLEncoder.encode(s, StandardCharsets.UTF_8);
}
}

```

dto/OtpDtos.java (new file)

```

package com.examly.springapp.dto;

/** Request and response shapes of the OTP endpoints. */
public final class OtpDtos {
    private OtpDtos() {}

    public record OtpSendRequest(String email) {}

    public record OtpVerifyRequest(String email, String code) {}

    /**
     * required = false means the platform does not ask for OTP (nothing was sent).
     * devCode is filled only in development mode with otp.dev-expose-code=true.
     */
    public record OtpSendResponse(boolean required, String message, int resendInSeconds, String devCode) {}

    public record OtpVerifyResponse(boolean verified, String message) {}
}

```

service/OtpService.java (new file)

```

package com.examly.springapp.service;

import com.examly.springapp.config.RateLimiter;
import com.examly.springapp.dto.OtpDtos.*;
import com.examly.springapp.exception.DatabaseOperationException;
import com.examly.springapp.exception.InvalidRequestException;
import com.examly.springapp.exception.PhoneNotVerifiedException;
import com.examly.springapp.exception.RateLimitExceededException;
import com.examly.springapp.model.PhoneVerification;
import com.examly.springapp.model.User;
import com.examly.springapp.repository.PhoneVerificationRepo;
import com.examly.springapp.repository.UserRepo;
import com.examly.springapp.sms.SmsSender;
import java.nio.charset.StandardCharsets;
import java.security.MessageDigest;
import java.security.SecureRandom;
import java.util.Base64;
import java.util.Locale;
import java.util.function.LongSupplier;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.core.env.Environment;
import org.springframework.core.env.Profiles;
import org.springframework.stereotype.Service;

/**
 * Mobile number verification with a one-time password (OTP).
 * <ul>
 * <li>A 6 digit code is made with SecureRandom, stored only as a salted SHA-256 hash, valid 5 minutes.
</li>
 * <li>5 wrong tries kill the code; a new code can be requested after 30 seconds, at most 5 per hour.</li>
 * <li>Answers never reveal whether an email exists or is waiting for verification.</li>
 * <li>When otp.required=false (the default) nothing is enforced, so existing flows and tests are
unchanged.</li>

```

```

* </ul>
*/
@Service
public class OtpService {
    private static final Logger log = LoggerFactory.getLogger(OtpService.class);
    static final long CODE_TTL_MS = 5 * 60_000L;
    static final int MAX_ATTEMPTS = 5;
    static final int RESEND_SECONDS = 30;
    private static final long HOUR_MS = 60 * 60_000L;
    static final String GENERIC_SEND = "If this account is waiting for verification, a code has been sent
to its mobile number.";

    private final PhoneVerificationRepo repo;
    private final UserRepo userRepo;
    private final SmsSender sms;
    private final RateLimiter limiter;
    private final boolean required;
    private final boolean exposeDevCode;
    private final String countryCode;
    private final LongSupplier clock;
    private final SecureRandom random = new SecureRandom();

    @Autowired
    public OtpService(PhoneVerificationRepo repo, UserRepo userRepo, SmsSender sms, RateLimiter limiter,
Environment env,
        @Value("${otp.required:false}") boolean required,
        @Value("${otp.dev-expose-code:false}") boolean exposeDevCode,
        @Value("${otp.country-code:+91}") String countryCode) {
        this(repo, userRepo, sms, limiter, required, exposeDevCode, countryCode,
System.currentTimeMillis);
        if (exposeDevCode && env.acceptsProfiles(Profiles.of("prod"))) {
            throw new IllegalStateException("Refusing to start: otp.dev-expose-code must be false in the
prod profile");
        }
    }

    OtpService(PhoneVerificationRepo repo, UserRepo userRepo, SmsSender sms, RateLimiter limiter, boolean
required,
        boolean exposeDevCode, String countryCode, LongSupplier clock) {
        this.repo = repo;
        this.userRepo = userRepo;
        this.sms = sms;
        this.limiter = limiter;
        this.required = required;
        this.exposeDevCode = exposeDevCode;
        this.countryCode = countryCode;
        this.clock = clock;
    }

    public boolean isRequired() {
        return required;
    }

    // ----- hooks used by registration and
login

    /** Called right after a customer or instructor registers: from now on the account must verify its
number. */
    public void markPending(User user) {
        if (!required || user == null || user.getUserId() == null) return;
        PhoneVerification row = DatabaseOperationException.guard("loading phone verification",
            () -> repo.findByUserId(user.getUserId()).orElseGet(() -> new
PhoneVerification(user.getUserId(), false)));
        row.setVerified(false);
        DatabaseOperationException.guardVoid("saving phone verification", () -> repo.save(row));
    }

    /** Called by login after the password was accepted: blocks accounts that still wait for verification.
*/
    public void requireVerified(User user) {
        if (!required || user == null) return;

```

```

        if (isPending(user.getUserId())) throw new PhoneNotVerifiedException();
    }

    /** True only when a row exists and says "not verified yet". */
    public boolean isPending(Long userId) {
        return DatabaseOperationException.guard("checking phone verification",
            () -> repo.findByUserId(userId)).map(r -> !r.isVerified()).orElse(false);
    }

    // ----- send

    public OtpSendResponse send(String emailRaw, String clientId) {
        if (!required) {
            return new OtpSendResponse(false, "Mobile verification is not switched on.", 0, null);
        }
        String email = emailRaw == null ? "" : emailRaw.trim().toLowerCase(Locale.ROOT);
        if (email.isEmpty() || email.length() > 120) {
            throw new InvalidRequestException("Email is required");
        }
        // limits apply to every email, existing or not, so the answers do not reveal which emails exist
        guardRate("otp:ip:" + clientId, 20, HOUR_MS, "Too many code requests from this device. Try again
later.");
        guardRate("otp:gap:" + email, 1, RESEND_SECONDS * 1000L, "Please wait before asking for another
code.");
        guardRate("otp:hour:" + email, 5, HOUR_MS, "Too many codes requested for this account. Try again in
an hour.");

        User user = DatabaseOperationException.guard("loading a user", () ->
userRepo.findByEmail(email)).orElse(null);
        PhoneVerification row = user == null ? null
            : DatabaseOperationException.guard("loading phone verification", () ->
repo.findByUserId(user.getUserId())).orElse(null);
        String devCode = null;
        if (user != null && row != null && !row.isVerified()) {
            String code = newCode();
            String salt = newSalt();
            row.setSalt(salt);
            row.setCodeHash(hash(salt, code));
            row.setAttempts(0);
            row.setLastSentAtMs(clock.getAsLong());
            row.setExpiresAtMs(clock.getAsLong() + CODE_TTL_MS);
            DatabaseOperationException.guardVoid("saving the code", () -> repo.save(row));
            sms.send(toE164(user.getMobileNumber()),
                "Your E-Learning Platform verification code is " + code + ". It expires in 5 minutes.
Do not share it.");
            log.info("OTP sent for user {}", user.getUserId());
            if (exposeDevCode) devCode = code;
        }
        return new OtpSendResponse(true, GENERIC_SEND, RESEND_SECONDS, devCode);
    }

    // ----- verify

    public OtpVerifyResponse verify(String emailRaw, String code, String clientId) {
        if (!required) {
            return new OtpVerifyResponse(true, "Mobile verification is not switched on.");
        }
        String email = emailRaw == null ? "" : emailRaw.trim().toLowerCase(Locale.ROOT);
        if (email.isEmpty() || code == null || !code.matches("^([0-9]){6}$")) {
            throw new InvalidRequestException("Enter the 6 digit code");
        }
        guardRate("otp:verify-ip:" + clientId, 30, HOUR_MS, "Too many attempts from this device. Try again
later.");

        User user = DatabaseOperationException.guard("loading a user", () ->
userRepo.findByEmail(email)).orElse(null);
        PhoneVerification row = user == null ? null
            : DatabaseOperationException.guard("loading phone verification", () ->
repo.findByUserId(user.getUserId())).orElse(null);
        if (row == null || row.isVerified() || row.getCodeHash() == null
            || clock.getAsLong() > row.getExpiresAtMs() || row.getAttempts() >= MAX_ATTEMPTS) {

```

```

        throw new InvalidRequestException("This code is wrong or has expired. Request a new code.");
    }
    boolean match = MessageDigest.isEqual(
        hash(row.getSalt(), code).getBytes(StandardCharsets.UTF_8),
        row.getCodeHash().getBytes(StandardCharsets.UTF_8));
    if (!match) {
        row.setAttempts(row.getAttempts() + 1);
        DatabaseOperationException.guardVoid("saving the attempt", () -> repo.save(row));
        int left = MAX_ATTEMPTS - row.getAttempts();
        throw new InvalidRequestException(left > 0
            ? "Wrong code. " + left + (left == 1 ? " try" : " tries") + " left."
            : "Too many wrong codes. Request a new code.");
    }
    row.setVerified(true);
    row.setCodeHash(null);
    row.setSalt(null);
    row.setAttempts(0);
    row.setVerifiedAtMs(clock.getAsLong());
    DatabaseOperationException.guardVoid("saving verification", () -> repo.save(row));
    log.info("Mobile number verified for user {}", user.getId());
    return new OtpVerifyResponse(true, "Your mobile number is verified. You can log in now.");
}

// ----- helpers

private void guardRate(String key, int limit, long windowMs, String message) {
    if (!limiter.tryAcquire(key, limit, windowMs)) {
        throw new RateLimitExceededException(message, limiter.retryAfterSeconds(key, windowMs));
    }
}

String toE164(String mobile) {
    String digits = mobile == null ? "" : mobile.replaceAll("[^0-9]", "");
    return countryCode + digits;
}

private String newCode() {
    return String.valueOf(100000 + random.nextInt(900000));
}

private String newSalt() {
    byte[] b = new byte[12];
    random.nextBytes(b);
    return Base64.getEncoder().withoutPadding().encodeToString(b);
}

static String hash(String salt, String code) {
    try {
        byte[] d = MessageDigest.getInstance("SHA-256").digest((salt + ":" +
        code).getBytes(StandardCharsets.UTF_8));
        return Base64.getEncoder().withoutPadding().encodeToString(d);
    } catch (java.security.NoSuchAlgorithmException e) {
        throw new IllegalStateException(e);
    }
}
}

```

controller/OtpController.java (new file)

```

package com.examly.springapp.controller;

import com.examly.springapp.dto.OtpDtos.*;
import com.examly.springapp.service.OtpService;
import io.swagger.v3.oas.annotations.Operation;
import io.swagger.v3.oas.annotations.tags.Tag;
import jakarta.servlet.http.HttpServletRequest;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

```

```

/** Mobile number verification by OTP. Public (the user cannot log in yet); abuse is limited inside
OtpService. */
@RestController
@RequestMapping("/api/otp")
@Tag(name = "Mobile OTP", description = "Send and check the verification code")
public class OtpController {
    private final OtpService otp;

    @Autowired
    public OtpController(OtpService otp) {
        this.otp = otp;
    }

    @Operation(summary = "Send a verification code",
        description = "Body: {email}. The code goes to the mobile number saved at registration. The
answer is the same for every email.")
    @PostMapping("/send")
    public ResponseEntity<OtpSendResponse> send(@RequestBody OtpSendRequest body, HttpServletRequest
request) {
        return ResponseEntity.ok(otp.send(body == null ? null : body.email(), request.getRemoteAddr()));
    }

    @Operation(summary = "Check the verification code", description = "Body: {email, code}. 6 digits, 5
tries, valid 5 minutes.")
    @PostMapping("/verify")
    public ResponseEntity<OtpVerifyResponse> verify(@RequestBody OtpVerifyRequest body, HttpServletRequest
request) {
        return ResponseEntity.ok(otp.verify(body == null ? null : body.email(), body == null ? null :
body.code(), request.getRemoteAddr()));
    }

    @Operation(summary = "Is OTP switched on?")
    @GetMapping("/status")
    public ResponseEntity<java.util.Map<String, Boolean>> status() {
        return ResponseEntity.ok(java.util.Map.of("required", otp.isRequired()));
    }
}

```